Java Palindrome Answer Book

All 113 palindrome questions with answers, Java snippets, complexity, and optimization notes.

1) Basic String Palindrome Questions

Question 1

Write a Java program to check whether a given string is a palindrome.

Answer

Use two pointers, one at the beginning and one at the end. Compare mirrored characters while moving inward. If a mismatch appears, the string is not a palindrome; otherwise it is.

Java Solution

```
boolean result = PalindromeSolutions.BasicStringSolutions.isPalindrome("level");
```

Complexity

Time O(n), Space O(1).

Optimized Approach

The two-pointer solution is already the optimized interview answer because it avoids building a reversed copy.

Question 2

Check if a string is a palindrome using the two-pointer approach.

Answer

The two-pointer approach compares the leftmost and rightmost characters, then shrinks the range toward the center. It is simple, fast, and works in constant extra space.

Java Solution

```
boolean result = PalindromeSolutions.BasicStringSolutions.isPalindrome("radar");
```

Complexity

Time O(n), Space O(1).

Optimized Approach

This is the optimized approach for ordinary string palindrome checking.

Question 3

Check if a string is a palindrome without using built-in reverse functions.

Answer

Avoid reverse helpers entirely and compare the string in place using indexes. That keeps the algorithm memory-efficient and easy to explain.

Java Solution

```
boolean result = PalindromeSolutions.BasicStringSolutions.isPalindrome("madam");
```

Complexity

Time O(n), Space O(1).

Optimized Approach

Compared with StringBuilder.reverse(), this avoids allocating another string and is more optimal in space.

Question 4

Check if a string is a palindrome using recursion.

Answer

Recursively compare the outer characters and continue with the inner substring. The recursion ends successfully once the left index crosses the right index.

Java Solution

```
boolean result = PalindromeSolutions.BasicStringSolutions.isPalindromeRecursive("level");
```

Complexity

Time O(n), Space O(n) due to recursion stack.

Optimized Approach

An optimized alternative is the iterative two-pointer method, which keeps the same time complexity but reduces space to O(1).

Question 5

Check if a string is a palindrome using a stack.

Answer

Push all characters onto a stack, then compare the original left-to-right order with the reverse order obtained by popping from the stack.

Java Solution

```
boolean result = PalindromeSolutions.BasicStringSolutions.isPalindromeWithStack("level");
```

Complexity

Time O(n), Space O(n).

Optimized Approach

The optimized alternative is the two-pointer method, which drops the stack and uses O(1) extra space.

Question 6

Check if a character array is a palindrome.

Answer

Treat the array exactly like a string and compare `char[]` values from both ends. Because arrays are mutable, this also works well when interviews avoid String helpers.

Java Solution

```
boolean result = PalindromeSolutions.BasicStringSolutions.isCharArrayPalindrome(
    new char[]{'r', 'a', 'd', 'a', 'r'}
);
```

Complexity

Time O(n), Space O(1).

Optimized Approach

The two-pointer scan is already optimal here.

Question 7

Return `true` if a string is a palindrome, otherwise return `false`.

Answer

This is just the boolean form of the standard palindrome check. The same mirrored comparison logic applies.

Java Solution

```
boolean result = PalindromeSolutions.BasicStringSolutions.isPalindrome("civic");
```

Complexity

Time O(n), Space O(1).

Optimized Approach

The optimized answer is still the in-place two-pointer comparison.

Question 8

Count how many words in a list are palindromes.

Answer

Iterate over the list and test each word with the basic palindrome checker. Increase the count whenever a word is palindromic.

Java Solution

```
long count = PalindromeSolutions.BasicStringSolutions.countPalindromeWords(
    List.of("level", "java", "madam")
);
```

Complexity

Time O(total characters across all words), Space O(1) extra.

Optimized Approach

If the list is very large, the main optimization is to reuse the same two-pointer logic on each word instead of constructing reversed copies.

2) Case, Spaces, and Special Character Variants

Question 9

Check whether a string is a palindrome ignoring case differences.

Answer

Convert mirrored characters to the same case before comparing them. This preserves the original string while making the comparison case-insensitive.

Java Solution

```
boolean result = PalindromeSolutions.NormalizedStringSolutions.isPalindromeIgnoreCase("Level");
```

Complexity

Time O(n), Space O(1).

Optimized Approach

The optimized approach lowercases only the characters being compared instead of creating a fully lowercased copy first.

Question 10

Check whether a sentence is a palindrome ignoring spaces.

Answer

Skip whitespace while moving the left and right pointers inward. Only non-space characters participate in the comparison.

Java Solution

```
boolean result = PalindromeSolutions.NormalizedStringSolutions.isPalindromeIgnoreSpaces(
    "n u r s e s r u n"
);
```

Complexity

Time O(n), Space O(1).

Optimized Approach

Skipping spaces during comparison is more space-efficient than building a cleaned intermediate string.

Question 11

Check whether a sentence is a palindrome ignoring spaces and punctuation.

Answer

Skip all characters that are not letters or digits, then compare the remaining meaningful characters in a case-insensitive way.

Java Solution

```
boolean result = PalindromeSolutions.NormalizedStringSolutions.isPalindromeAlphaNumeric(
    "Able was I, ere I saw Elba."
);
```

Complexity

Time O(n), Space O(1).

Optimized Approach

The optimized method avoids generating a sanitized string and instead filters characters on the fly.

Question 12

Check whether a string is a palindrome by considering only alphanumeric characters.

Answer

This is the classic valid-palindrome problem. Ignore every non-alphanumeric character and compare the rest from both ends.

Java Solution

```
boolean result = PalindromeSolutions.NormalizedStringSolutions.isPalindromeAlphaNumeric(
    "A man, a plan, a canal: Panama"
);
```

Complexity

Time O(n), Space O(1).

Optimized Approach

On-the-fly filtering is the optimized approach compared with building a cleaned copy first.

Question 13

Check whether a Unicode string is a palindrome after normalizing letter case.

Answer

Normalize the input to a standard Unicode form, convert it to lowercase, then compare code points rather than raw UTF-16 code units so the logic handles broader character sets correctly.

Java Solution

```
boolean result = PalindromeSolutions.NormalizedStringSolutions.isUnicodeNormalizedPalindrome("Level");
```

Complexity

Time O(n), Space O(n).

Optimized Approach

An optimized interview answer may skip full Unicode handling unless the question explicitly asks for it. For ASCII-only input, the simpler two-pointer lowercase comparison is lighter.

Question 14

Validate if a phrase like `"A man, a plan, a canal: Panama"` is a palindrome.

Answer

Ignore case, spaces, and punctuation so that only the important letters and digits are compared in mirrored positions.

Java Solution

```
boolean result = PalindromeSolutions.NormalizedStringSolutions.isPalindromeAlphaNumeric(
    "A man, a plan, a canal: Panama"
);
```

Complexity

Time O(n), Space O(1).

Optimized Approach

This is already the optimized practical approach for phrase-style palindrome validation.

3) Constraint-Based String Questions

Question 15

Check if a string is a palindrome in `O(1)` extra space.

Answer

Use only indexes and compare the original string directly. No auxiliary stack, array, or reversed string is needed.

Java Solution

```
boolean result = PalindromeSolutions.ConstraintStringSolutions.isPalindromeO1Space("racecar");
```

Complexity

Time O(n), Space O(1).

Optimized Approach

This question explicitly asks for the optimized approach, and the two-pointer scan is that answer.

Question 16

Check if a string is a palindrome without creating another string.

Answer

The simplest way is to compare the existing string in place using left and right indexes. That avoids copy construction entirely.

Java Solution

```
boolean result = PalindromeSolutions.ConstraintStringSolutions.isPalindromeO1Space("abba");
```

Complexity

Time O(n), Space O(1).

Optimized Approach

This is more optimal than reversing with StringBuilder because no new string object is created.

Question 17

Check if a string is a palindrome using Java Streams.

Answer

Generate the left-half indexes with a stream and verify that every position matches its mirrored partner on the right.

Java Solution

```
boolean result = PalindromeSolutions.ConstraintStringSolutions.isPalindromeStream("level");
```

Complexity

Time O(n), Space O(1) extra.

Optimized Approach

The stream solution is concise, but the optimized interview version is still the standard loop-based two-pointer solution.

Question 18

Check if a string is a palindrome using `StringBuilder`, then discuss why it is less optimal.

Answer

Reverse the string with `StringBuilder` and compare it with the original. This is readable but costs extra memory because it builds another string representation.

Java Solution

```
boolean result = PalindromeSolutions.ConstraintStringSolutions.isPalindromeWithStringBuilder("level");
```

Complexity

Time O(n), Space O(n).

Optimized Approach

The optimized approach is the two-pointer method, which keeps the same time complexity but reduces space to O(1).

Question 19

Check if only a substring from index `l` to `r` is a palindrome.

Answer

Restrict the comparison to the requested range and compare characters symmetrically inside that window only.

Java Solution

```
boolean result = PalindromeSolutions.ConstraintStringSolutions.isSubstringPalindrome(
    "abacaba", 1, 5
);
```

Complexity

Time O(r - l + 1), Space O(1).

Optimized Approach

For many repeated queries, an optimized approach is preprocessing with rolling hash or DP instead of checking every range from scratch.

Question 20

Check if a string can become a palindrome after removing at most one character.

Answer

Scan from both ends. At the first mismatch, try skipping the left character once or the right character once; if either remaining range is palindromic, the answer is true.

Java Solution

```
boolean result = PalindromeSolutions.ConstraintStringSolutions
    .canBecomePalindromeAfterRemovingAtMostOne("abca");
```

Complexity

Time O(n), Space O(1).

Optimized Approach

This is already the optimized linear solution for the one-removal variant.

Question 21

Check if a string can become a palindrome after removing exactly one character.

Answer

Try removing each index and test whether the remaining characters form a palindrome. Return true as soon as one removal works.

Java Solution

```
boolean result = PalindromeSolutions.ConstraintStringSolutions
    .canBecomePalindromeAfterRemovingExactlyOne("abca");
```

Complexity

Time O(n^2), Space O(1).

Optimized Approach

An optimized approach can be derived from the at-most-one-removal two-pointer idea by handling whether the original string is already a palindrome and then checking mismatch positions directly instead of trying every index.

Question 22

Find the first index whose removal makes the string a palindrome, if possible.

Answer

Check each possible removed position from left to right and return the first one that makes the remaining characters palindromic.

Java Solution

```
int index = PalindromeSolutions.ConstraintStringSolutions
    .firstRemovalIndexForPalindrome("cabba");
```

Complexity

Time O(n^2), Space O(1).

Optimized Approach

A more optimized approach can focus only on mismatch positions found by a two-pointer scan instead of testing every index.

4) Number Palindrome Questions

Question 23

Write a Java program to check whether an integer is a palindrome.

Answer

Convert the integer to a string and apply the basic string palindrome check. This is easy to explain and suitable for simple interviews.

Java Solution

```
boolean result = PalindromeSolutions.NumberPalindromeSolutions.isPalindromeInt(12321);
```

Complexity

Time O(d), Space O(d), where d is the number of digits.

Optimized Approach

The optimized approach is question 24: compare the numeric halves without converting to a string.

Question 24

Check if a number is a palindrome without converting it to a string.

Answer

Reverse only the last half of the number and compare it with the first half. This prevents full reversal overflow and keeps memory usage constant.

Java Solution

```
boolean result = PalindromeSolutions.NumberPalindromeSolutions.isPalindromeWithoutString(12321);
```

Complexity

Time O(d), Space O(1).

Optimized Approach

This is the optimized approach over string conversion.

Question 25

Check if a long integer is a palindrome.

Answer

Apply the same half-reversal idea as with `int`, but on `long` values.

Java Solution

```
boolean result = PalindromeSolutions.NumberPalindromeSolutions.isPalindromeLong(123454321L);
```

Complexity

Time O(d), Space O(1).

Optimized Approach

Half reversal remains the optimized method here too.

Question 26

Reverse an integer safely and use it to test whether the original number is a palindrome.

Answer

Reverse digit by digit using a wider numeric type to detect overflow. If reversing succeeds, compare the result with the original integer.

Java Solution

```
OptionalInt reversed = PalindromeSolutions.NumberPalindromeSolutions.reverseIntSafely(12321);
boolean result = reversed.isPresent() && reversed.getAsInt() == 12321;
```

Complexity

Time O(d), Space O(1).

Optimized Approach

The optimized palindrome-only version is half reversal from question 24 because it avoids reversing the entire number at all.

Question 27

Check whether a negative number should be treated as a palindrome and justify the rule.

Answer

Under the usual numeric interpretation, negative numbers are not palindromes because the minus sign appears only on the left side and cannot be mirrored on the right.

Java Solution

```
boolean result = PalindromeSolutions.NumberPalindromeSolutions.isPalindromeWithNegativeRule(-121);
```

Complexity

Time O(d), Space O(1).

Optimized Approach

The optimized behavior is simply to reject negatives early before doing any other work.

Question 28

Check whether a number is a palindrome in binary representation.

Answer

Identify the highest set bit and compare it with the lowest bit, moving inward until all mirrored binary positions are checked.

Java Solution

```
boolean result = PalindromeSolutions.NumberPalindromeSolutions.isBinaryPalindrome(9);
```

Complexity

Time O(log n), Space O(1).

Optimized Approach

Bit comparison is more optimized than constructing a binary string and reversing it.

Question 29

Check whether a number is a palindrome in any given base `b`.

Answer

Extract the digits in base `b`, store them, and compare the digit sequence from both ends.

Java Solution

```
boolean result = PalindromeSolutions.NumberPalindromeSolutions.isPalindromeInBase(585, 2);
```

Complexity

Time O(log_b n), Space O(log_b n).

Optimized Approach

For very strict space constraints, an optimized solution can compare leading and trailing digits mathematically without storing the whole representation, though that is more complex to implement.

Question 30

Count all palindrome numbers in a range `[L, R]`.

Answer

Iterate from `L` to `R` and test each value with the numeric palindrome method. This is straightforward and correct for moderate ranges.

Java Solution

```
long count = PalindromeSolutions.NumberPalindromeSolutions
    .countPalindromeNumbersInRange(1, 200);
```

Complexity

Time O((R - L + 1) * log R), Space O(1).

Optimized Approach

For large ranges, the optimized strategy is to generate palindromes directly instead of testing every number.

Question 31

Generate all palindrome numbers with `n` digits.

Answer

Generate only the first half of each palindrome and mirror it to form the full number. This avoids scanning all n-digit numbers.

Java Solution

```
List<Long> values = PalindromeSolutions.NumberPalindromeSolutions.generateNDigitPalindromes(3);
```

Complexity

Time proportional to the number of generated palindromes, O(10^(ceil(n / 2))). Space is the same including output.

Optimized Approach

Half-generation plus mirroring is the optimized approach.

Question 32

Find the next palindrome number greater than a given integer.

Answer

The current implementation increments the number until a palindrome is found. It is simple but may be slow if the gap is large.

Java Solution

```
long next = PalindromeSolutions.NumberPalindromeSolutions.nextPalindromeNumber(123);
```

Complexity

Current implementation: Time O(gap * log n), Space O(1).

Optimized Approach

An optimized approach mirrors the left half of the number and, if needed, increments the middle before mirroring again.

Question 33

Find the nearest palindrome number to a given integer.

Answer

Search outward on both sides until a palindrome is discovered. The current implementation returns the lower palindrome first if both sides are equally close.

Java Solution

```
long nearest = PalindromeSolutions.NumberPalindromeSolutions.nearestPalindromeNumber(123);
```

Complexity

Current implementation: Time O(gap * log n), Space O(1).

Optimized Approach

An optimized approach generates a small candidate set by mirroring the number and nearby prefixes, then picks the closest candidate.

5) Array and Collection Palindrome Questions

Question 34

Check whether an integer array is a palindrome.

Answer

Compare the first and last elements, then move inward. If every mirrored pair matches, the array is a palindrome.

Java Solution

```
boolean result = PalindromeSolutions.ArrayCollectionSolutions
    .isArrayPalindrome(new int[]{1, 2, 3, 2, 1});
```

Complexity

Time O(n), Space O(1).

Optimized Approach

This two-pointer array scan is already optimal.

Question 35

Check whether a generic `List` is a palindrome.

Answer

Compare elements at symmetric positions using `Objects.equals`, which handles nulls safely.

Java Solution

```
boolean result = PalindromeSolutions.ArrayCollectionSolutions
    .isListPalindrome(List.of("a", "b", "a"));
```

Complexity

Time O(n), Space O(1) for random-access lists.

Optimized Approach

For linked lists, the optimized technique differs and usually involves a stack or list reversal.

Question 36

Check whether an array is a palindrome in-place using two pointers.

Answer

Use the same array directly and do not create any copy. Just compare mirrored positions in the original array.

Java Solution

```
boolean result = PalindromeSolutions.ArrayCollectionSolutions
    .isArrayPalindrome(new int[]{4, 5, 5, 4});
```

Complexity

Time O(n), Space O(1).

Optimized Approach

This is the optimized in-place approach.

Question 37

Determine whether a deque of characters forms a palindrome.

Answer

Remove one character from the front and one from the back and compare them. Continue until the deque size is zero or one.

Java Solution

```
boolean result = PalindromeSolutions.ArrayCollectionSolutions
    .isDequePalindrome(new ArrayDeque<>(List.of('n', 'o', 'o', 'n')));
```

Complexity

Time O(n), Space O(n) in the current implementation because it copies the deque.

Optimized Approach

An optimized variant can mutate the original deque directly and reduce extra space to O(1) if preserving the input is not required.

Question 38

Check whether a list of strings is a palindrome by value comparison.

Answer

This is a specialization of the generic list palindrome problem: compare the first string with the last, second with second-last, and so on.

Java Solution

```
boolean result = PalindromeSolutions.ArrayCollectionSolutions
    .isListPalindrome(List.of("red", "blue", "red"));
```

Complexity

Time O(n), Space O(1) for random-access lists.

Optimized Approach

The same optimized two-index logic applies.

Question 39

Count how many subarrays of length `k` are palindromic.

Answer

Slide a window of length `k` across the array and test each window with a two-pointer check.

Java Solution

```
long count = PalindromeSolutions.ArrayCollectionSolutions
    .countPalindromicSubarrays(new int[]{1, 2, 1, 2, 1}, 3);
```

Complexity

Time O((n - k + 1) * k), Space O(1).

Optimized Approach

For many queries or very large arrays, an optimized approach could use rolling hash or Manacher-style ideas adapted to arrays.

Question 40

Find the longest palindromic subarray in an integer array.

Answer

Expand around every possible single center and every gap center exactly like longest palindromic substring, but on integers instead of characters.

Java Solution

```
int[] best = PalindromeSolutions.ArrayCollectionSolutions
    .longestPalindromicSubarray(new int[]{1, 2, 3, 2, 1, 9});
```

Complexity

Time O(n^2), Space O(1) extra.

Optimized Approach

The center-expansion technique is a strong optimized interview answer compared with brute-force checking of all subarrays.

6) Linked List Palindrome Questions

Question 41

Check whether a singly linked list is a palindrome.

Answer

Find the midpoint, reverse the second half, compare both halves, and then optionally restore the original list.

Java Solution

```
boolean result = PalindromeSolutions.LinkedListSolutions.isPalindromeAndRestore(
    PalindromeSolutions.LinkedListSolutions.SinglyNode.of(1, 2, 3, 2, 1)
);
```

Complexity

Time O(n), Space O(1).

Optimized Approach

This reverse-second-half technique is the optimized linked-list approach.

Question 42

Check whether a singly linked list is a palindrome using a stack.

Answer

Traverse the list once to push node values, then compare them during a second traversal.

Java Solution

```
boolean result = PalindromeSolutions.LinkedListSolutions.isPalindromeWithStack(
    PalindromeSolutions.LinkedListSolutions.SinglyNode.of(1, 2, 1)
);
```

Complexity

Time O(n), Space O(n).

Optimized Approach

The optimized alternative is reversing the second half to reduce extra space to O(1).

Question 43

Check whether a singly linked list is a palindrome by reversing the second half.

Answer

Use slow and fast pointers to reach the middle, reverse the tail portion, and compare it with the first half.

Java Solution

```
boolean result = PalindromeSolutions.LinkedListSolutions.isPalindromeByReversingSecondHalf(
    PalindromeSolutions.LinkedListSolutions.SinglyNode.of(1, 2, 2, 1)
);
```

Complexity

Time O(n), Space O(1).

Optimized Approach

This is the optimized solution compared with stack-based checking.

Question 44

Restore the original linked list after palindrome checking.

Answer

After the comparison, reverse the second half again and reconnect it so the input list keeps its initial shape.

Java Solution

```
boolean result = PalindromeSolutions.LinkedListSolutions.isPalindromeAndRestore(
    PalindromeSolutions.LinkedListSolutions.SinglyNode.of(1, 2, 3, 2, 1)
);
```

Complexity

Time O(n), Space O(1).

Optimized Approach

Restoring the list is the polished optimized interview answer when mutation of input is a concern.

Question 45

Check whether a doubly linked list is a palindrome.

Answer

Start one pointer at the head and one at the tail, then compare values while moving both toward the center.

Java Solution

```
boolean result = PalindromeSolutions.LinkedListSolutions.isDoublyListPalindrome(
    PalindromeSolutions.LinkedListSolutions.DoublyNode.of(1, 2, 3, 2, 1)
);
```

Complexity

Time O(n), Space O(1).

Optimized Approach

Because a doubly linked list gives backward access, this direct two-ended scan is already optimal.

Question 46

Check whether a circular linked list is a palindrome under one full traversal.

Answer

Read exactly one cycle into a linear sequence of known length, then apply the usual palindrome check to that collected data.

Java Solution

```
boolean result = PalindromeSolutions.LinkedListSolutions.isCircularListPalindrome(
    PalindromeSolutions.LinkedListSolutions.CircularNode.of(1, 2, 1), 3
);
```

Complexity

Time O(n), Space O(n).

Optimized Approach

A more optimized approach may use specialized pointer logic, but collecting one full traversal is a clean and reliable answer.

7) Substring-Based Palindrome Questions

Question 47

Find the longest palindromic substring in a string.

Answer

Expand around every possible center and record the best range. Each palindrome is centered either at a character or between two characters.

Java Solution

```
String best = PalindromeSolutions.SubstringSolutions
    .longestPalindromicSubstring("forgeeksskeegfor");
```

Complexity

Time O(n^2), Space O(1).

Optimized Approach

Center expansion is the optimized general-purpose interview solution. Manacher's algorithm can improve time to O(n) but is harder to explain.

Question 48

Count all palindromic substrings in a string.

Answer

Expand around every center and count each successful expansion as one palindrome.

Java Solution

```
long count = PalindromeSolutions.SubstringSolutions.countPalindromicSubstrings("aaa");
```

Complexity

Time O(n^2), Space O(1).

Optimized Approach

Center expansion is a strong optimized answer compared with brute-force checking of every substring.

Question 49

Print all distinct palindromic substrings of a string.

Answer

Expand around centers and store each found substring in a set so duplicate palindromes appear only once.

Java Solution

```
Set<String> values = PalindromeSolutions.SubstringSolutions
    .distinctPalindromicSubstrings("ababa");
```

Complexity

Worst-case time O(n^3) with substring materialization, Space O(n^2) worst case.

Optimized Approach

If only counting is needed, avoid materializing substrings. If distinct reporting is required, using a set is a practical approach.

Question 50

Find the length of the longest palindromic substring.

Answer

Run the longest-palindromic-substring algorithm and return the length instead of the actual substring.

Java Solution

```
int length = PalindromeSolutions.SubstringSolutions
    .lengthOfLongestPalindromicSubstring("babad");
```

Complexity

Time O(n^2), Space O(1).

Optimized Approach

The optimized approach is still center expansion, unless the interview specifically expects Manacher's algorithm.

Question 51

Return the starting index of the longest palindromic substring.

Answer

Track the best palindrome range while expanding around centers and return the left boundary of that best range.

Java Solution

```
int index = PalindromeSolutions.SubstringSolutions
    .startIndexOfLongestPalindromicSubstring("forgeeksskeegfor");
```

Complexity

Time O(n^2), Space O(1).

Optimized Approach

Center expansion remains the optimized answer for readability and simplicity.

Question 52

Find the longest even-length palindromic substring.

Answer

Only expand around gaps between adjacent characters, because even-length palindromes have no single central character.

Java Solution

```
String best = PalindromeSolutions.SubstringSolutions
    .longestEvenPalindromicSubstring("abccba");
```

Complexity

Time O(n^2), Space O(1).

Optimized Approach

Restricting to gap centers is the optimized specialization of the center-expansion method.

Question 53

Find the longest odd-length palindromic substring.

Answer

Only expand around actual characters because odd-length palindromes have a single center element.

Java Solution

```
String best = PalindromeSolutions.SubstringSolutions
    .longestOddPalindromicSubstring("bananas");
```

Complexity

Time O(n^2), Space O(1).

Optimized Approach

Restricting to character centers is the optimized specialized version of center expansion.

Question 54

Find all palindromic substrings longer than length `k`.

Answer

Expand around all centers and keep only the palindromes whose length exceeds `k`.

Java Solution

```
List<String> values = PalindromeSolutions.SubstringSolutions
    .palindromicSubstringsLongerThan("abacdcaba", 2);
```

Complexity

Worst-case time O(n^3) with substring construction, Space O(output).

Optimized Approach

If only counts are required, a more optimized approach avoids building the substring objects themselves.

Question 55

Count palindromic substrings for multiple test cases efficiently.

Answer

Apply the same center-expansion logic independently to each test case. This is practical for moderate input sizes and easy to implement correctly.

Java Solution

```
List<Long> counts = testCases.stream()
    .map(PalindromeSolutions.SubstringSolutions::countPalindromicSubstrings)
    .toList();
```

Complexity

Time is the sum of O(length^2) for each test case, Space O(1) extra per case.

Optimized Approach

For very large numbers of repeated queries on the same string, preprocessing approaches such as Manacher's algorithm or query-specific data structures become the optimized option.

8) Subsequence and DP-Based Questions

Question 56

Find the length of the longest palindromic subsequence.

Answer

Define `dp[i][j]` as the longest palindromic subsequence length in the substring from `i` to `j`. If both ends match, include them; otherwise drop one end and take the better answer.

Java Solution

```
int length = PalindromeSolutions.DynamicProgrammingSolutions
    .longestPalindromicSubsequenceLength("bbbab");
```

Complexity

Time O(n^2), Space O(n^2).

Optimized Approach

Dynamic programming is the optimized standard approach over brute-force subsequence enumeration.

Question 57

Print one longest palindromic subsequence.

Answer

First fill the DP table for LPS length, then walk the table from both ends to reconstruct one valid subsequence.

Java Solution

```
String lps = PalindromeSolutions.DynamicProgrammingSolutions
    .oneLongestPalindromicSubsequence("bbbab");
```

Complexity

Time O(n^2), Space O(n^2).

Optimized Approach

The optimized strategy is to reconstruct from the DP table rather than enumerate all subsequences.

Question 58

Count the number of palindromic subsequences in a string.

Answer

Use DP to count answers for smaller ranges and combine them carefully. When the boundary characters differ, subtract overlap to avoid double-counting.

Java Solution

```
long count = PalindromeSolutions.DynamicProgrammingSolutions
    .countPalindromicSubsequences("aaa");
```

Complexity

Time O(n^2), Space O(n^2).

Optimized Approach

DP is the optimized approach; brute force over all subsequences is exponential.

Question 59

Find the minimum number of insertions needed to make a string a palindrome.

Answer

The answer equals `n - LPS`, because every character not belonging to an optimal palindromic subsequence must be inserted somewhere to mirror another character.

Java Solution

```
int answer = PalindromeSolutions.DynamicProgrammingSolutions
    .minInsertionsToPalindrome("abcda");
```

Complexity

Time O(n^2), Space O(n^2).

Optimized Approach

Using the longest palindromic subsequence relation is the optimized dynamic-programming insight.

Question 60

Find the minimum number of deletions needed to make a string a palindrome.

Answer

This is symmetric to minimum insertions: remove every character that is not part of a longest palindromic subsequence.

Java Solution

```
int answer = PalindromeSolutions.DynamicProgrammingSolutions
    .minDeletionsToPalindrome("abcda");
```

Complexity

Time O(n^2), Space O(n^2).

Optimized Approach

The optimized insight is again `n - LPS`.

Question 61

Find the minimum number of replacements needed to make a string a palindrome.

Answer

Walk inward from both ends. Every mismatched mirrored pair can be fixed with exactly one replacement.

Java Solution

```
int answer = PalindromeSolutions.DynamicProgrammingSolutions
    .minReplacementsToPalindrome("abcdef");
```

Complexity

Time O(n), Space O(1).

Optimized Approach

Counting mismatched pairs directly is already optimal.

Question 62

Determine whether a string is a `k`-palindrome.

Answer

A string is a `k`-palindrome if it can be made palindromic after deleting at most `k` characters. Compute minimum deletions or equivalently use the LPS relation.

Java Solution

```
boolean result = PalindromeSolutions.DynamicProgrammingSolutions
    .isKPalindrome("abcdecba", 1);
```

Complexity

Time O(n^2), Space O(n^2).

Optimized Approach

The optimized approach is dynamic programming instead of trying all deletion combinations.

Question 63

Partition a string into palindromic substrings using dynamic programming.

Answer

Precompute which ranges are palindromes, then backtrack using only valid palindromic pieces to build all partitions.

Java Solution

```
List<List<String>> parts = PalindromeSolutions.DynamicProgrammingSolutions
    .palindromePartitions("aab");
```

Complexity

Time O(n^2 + output_size), Space O(n^2 + output_size).

Optimized Approach

The optimized idea is to separate palindrome preprocessing from partition generation.

Question 64

Find the minimum cuts needed for palindrome partitioning.

Answer

Use a palindrome table and a `cuts[i]` array that stores the minimum cuts needed for the prefix ending at `i`. If a suffix is palindromic, it can serve as the final partition segment.

Java Solution

```
int cuts = PalindromeSolutions.DynamicProgrammingSolutions
    .minCutPalindromePartition("aab");
```

Complexity

Time O(n^2), Space O(n^2).

Optimized Approach

This DP is the optimized answer over brute-force partitioning.

Question 65

Count the number of ways to partition a string so every part is a palindrome.

Answer

After precomputing palindrome ranges, count how many valid partitions start at each index using dynamic programming from right to left.

Java Solution

```
long ways = PalindromeSolutions.DynamicProgrammingSolutions
    .countPalindromePartitions("aab");
```

Complexity

Time O(n^2), Space O(n^2).

Optimized Approach

Dynamic programming is the optimized solution; brute force quickly becomes exponential.

9) Rearrangement and Construction Questions

Question 66

Check whether any permutation of a string can form a palindrome.

Answer

Count character frequencies. A palindrome permutation is possible only if at most one character appears an odd number of times.

Java Solution

```
boolean result = PalindromeSolutions.RearrangementConstructionSolutions
    .canPermutePalindrome("carrace");
```

Complexity

Time O(n log sigma) in the current TreeMap implementation, Space O(sigma).

Optimized Approach

The optimized logic is frequency parity checking; using a HashMap can reduce the map overhead to near O(n).

Question 67

Rearrange the characters of a string to form one palindrome, if possible.

Answer

Build half of the palindrome from paired characters, keep one odd character for the center, and mirror the half to the other side.

Java Solution

```
String value = PalindromeSolutions.RearrangementConstructionSolutions
    .buildOnePalindrome("aabbccd");
```

Complexity

Time O(n log sigma), Space O(sigma).

Optimized Approach

The optimized construction uses counts instead of trying permutations.

Question 68

Generate all palindromic permutations of a string.

Answer

Generate unique permutations of only the half-string and mirror each result. This is much better than generating all permutations of the full string and filtering them.

Java Solution

```
List<String> values = PalindromeSolutions.RearrangementConstructionSolutions
    .generatePalindromicPermutations("aabb");
```

Complexity

Time O(p * n), Space O(p * n), where p is the number of generated palindromes.

Optimized Approach

Generating permutations of the half-string is the optimized approach compared with full-permutation brute force.

Question 69

Build the longest possible palindrome using the characters of a string.

Answer

Take all even counts and one odd center if available. Every other odd leftover contributes only its largest even part.

Java Solution

```
int length = PalindromeSolutions.RearrangementConstructionSolutions
    .longestPossiblePalindromeLength("abcccdd");
```

Complexity

Time O(n log sigma), Space O(sigma).

Optimized Approach

The optimized insight is to use all possible pairs and only one odd center.

Question 70

Build the lexicographically smallest palindrome from the given characters.

Answer

Sort or store characters in ascending order, build the left half from the smallest available pairs, place the middle if needed, then mirror it.

Java Solution

```
String value = PalindromeSolutions.RearrangementConstructionSolutions
    .lexicographicallySmallestPalindrome("aabbccd");
```

Complexity

Time O(n log sigma), Space O(sigma).

Optimized Approach

Using ordered frequency storage is the optimized construction for lexicographic requirements.

Question 71

Determine the minimum swaps needed to rearrange a string into a palindrome.

Answer

For arbitrary swaps, first choose a target palindrome arrangement, then compute the minimum swaps needed to transform the source arrangement into that target using index mapping and cycle decomposition.

Java Solution

```
String target = buildTargetPalindrome(value);
int swaps = minArbitrarySwaps(source, target);
```

Complexity

Usually O(n log n) to O(n^2) depending on duplicate handling, Space O(n).

Optimized Approach

Compared with adjacent-swap simulation, arbitrary-swap counting can be more efficient when the interview allows swapping any two positions.

Question 72

Determine the minimum adjacent swaps needed to make a string a palindrome.

Answer

Move matching characters toward the left and right ends by repeated adjacent swaps. When a character has no partner, it must be the middle character and is bubbled inward.

Java Solution

```
int swaps = PalindromeSolutions.RearrangementConstructionSolutions
    .minAdjacentSwapsToMakePalindrome("mamad");
```

Complexity

Time O(n^2), Space O(1).

Optimized Approach

This greedy adjacent-swap strategy is the standard optimized answer for the adjacent-only version.

Question 73

Add the minimum number of characters to the end of a string to make it a palindrome.

Answer

Find the earliest suffix that is already a palindrome. All preceding characters must be appended in reverse order to the end.

Java Solution

```
int count = PalindromeSolutions.RearrangementConstructionSolutions
    .minCharsToAppendAtEnd("abcd");
String shortest = PalindromeSolutions.RearrangementConstructionSolutions
    .shortestPalindromeByAppendingEnd("abcd");
```

Complexity

Current implementation: Time O(n^2), Space O(1) extra for the count.

Optimized Approach

A more optimized version can use rolling hash or prefix-function style preprocessing to reduce repeated suffix checking.

Question 74

Add the minimum number of characters to the front of a string to make it a palindrome.

Answer

Find the longest palindromic prefix. Everything after that prefix must be reversed and added in front.

Java Solution

```
int count = PalindromeSolutions.RearrangementConstructionSolutions
    .minCharsToAddInFront("aacecaaa");
```

Complexity

Time O(n), Space O(n).

Optimized Approach

Using the longest-palindromic-prefix idea with KMP-style preprocessing is the optimized approach.

Question 75

Construct the shortest palindrome by adding characters in front of the string.

Answer

This is the constructive form of question 74. Reverse the suffix that lies outside the longest palindromic prefix and prepend it.

Java Solution

```
String value = PalindromeSolutions.RearrangementConstructionSolutions
    .shortestPalindromeByAddingFront("abcd");
```

Complexity

Time O(n), Space O(n).

Optimized Approach

The KMP/prefix-function-based prefix detection is the optimized approach here.

10) Pairing and Combination Questions

Question 76

Given a list of words, find all pairs whose concatenation is a palindrome.

Answer

For each word, split it into every possible prefix and suffix. If one side is a palindrome, the reversed other side may be the needed partner in a hash map.

Java Solution

```
List<List<Integer>> pairs = PalindromeSolutions.PairingCombinationSolutions
    .palindromePairs(new String[]{"bat", "tab", "cat"});
```

Complexity

Time O(n * k^2), Space O(n * k).

Optimized Approach

Using a hash map for reverse lookups is the optimized approach compared with testing every pair directly in O(n^2 * k).

Question 77

Count the number of palindrome pairs in an array of strings.

Answer

Generate all palindrome pairs using the same prefix-suffix split method and count them.

Java Solution

```
int count = PalindromeSolutions.PairingCombinationSolutions
    .countPalindromePairs(new String[]{"bat", "tab", "cat"});
```

Complexity

Time O(n * k^2), Space O(n * k).

Optimized Approach

The optimized strategy is the same hash-map-based palindrome pair search.

Question 78

Check whether two given strings can be combined to form a palindrome.

Answer

Try both concatenation orders, because `a + b` and `b + a` are different candidates.

Java Solution

```
boolean result = PalindromeSolutions.PairingCombinationSolutions
    .canCombineToPalindrome("abc", "cba");
```

Complexity

Time O(n + m), Space O(n + m) in the current implementation.

Optimized Approach

A more optimized low-level implementation could compare characters without materializing the concatenated strings.

Question 79

Find the longest palindrome that can be formed by concatenating two strings.

Answer

The repository implementation concatenates the strings in both orders and finds the longest palindromic substring within each concatenation, returning the better one.

Java Solution

```
String best = PalindromeSolutions.PairingCombinationSolutions
    .longestPalindromeFromConcatenatingTwoStrings("abaxy", "zyxf");
```

Complexity

Time O((n + m)^2), Space O(1) extra beyond produced substrings.

Optimized Approach

An optimized interview solution depends on the exact problem interpretation. Some variants use DP or center expansion over constrained cross-boundary joins.

Question 80

Build the longest palindrome from a list of two-letter words.

Answer

Pair each word with its reverse to contribute four characters. Symmetric words like `"gg"` can also place one leftover word in the center.

Java Solution

```
int length = PalindromeSolutions.PairingCombinationSolutions
    .longestPalindromeFromTwoLetterWords(new String[]{"lc", "cl", "gg"});
```

Complexity

Time O(w), Space O(w), where w is the number of words.

Optimized Approach

Counting frequencies in a hash map is the optimized approach.

11) Matrix, Grid, and Pattern Variants

Question 81

Check whether each row of a character matrix is a palindrome.

Answer

Treat each row as a `char[]` palindrome check and count or report which rows succeed.

Java Solution

```
long rows = PalindromeSolutions.MatrixGridPatternSolutions.countPalindromicRows(matrix);
```

Complexity

Time O(r * c), Space O(1).

Optimized Approach

This row-wise direct scan is already optimal.

Question 82

Check whether each column of a matrix is a palindrome.

Answer

For each column, compare the top cell with the bottom cell, then move inward until the center is reached.

Java Solution

```
long cols = PalindromeSolutions.MatrixGridPatternSolutions.countPalindromicColumns(matrix);
```

Complexity

Time O(r * c), Space O(1).

Optimized Approach

This vertical mirrored scan is already optimal.

Question 83

Count palindromic rows and palindromic columns in a matrix.

Answer

Count row palindromes and column palindromes independently, then add the results.

Java Solution

```
long total = PalindromeSolutions.MatrixGridPatternSolutions
    .countPalindromicRowsAndColumns(matrix);
```

Complexity

Time O(r * c), Space O(1).

Optimized Approach

No better asymptotic approach is needed because every cell must be part of some row or column check.

Question 84

Determine whether a path string formed in a grid is a palindrome.

Answer

Once the path labels are collected into a string, the problem reduces to the basic string palindrome check.

Java Solution

```
boolean result = PalindromeSolutions.MatrixGridPatternSolutions
    .isGridPathPalindrome("abccba");
```

Complexity

Time O(path length), Space O(1) extra after the path string is built.

Optimized Approach

If path enumeration itself is part of the task, optimized graph traversal becomes the real challenge rather than the palindrome check.

Question 85

Find all palindromic diagonals in a square matrix.

Answer

Collect diagonal strings in both major directions and keep only those that read the same forward and backward.

Java Solution

```
List<String> values = PalindromeSolutions.MatrixGridPatternSolutions
    .palindromicDiagonals(matrix);
```

Complexity

Current implementation: Time O(r * c * min(r, c)), Space O(output).

Optimized Approach

An optimized approach could avoid rebuilding diagonal strings repeatedly by streaming comparisons directly from both ends of each diagonal.

12) Hashing, Queries, and Large Input Questions

Question 86

Answer multiple palindrome substring queries efficiently.

Answer

Preprocess the string once so each query can be answered quickly. The repository uses a rolling-hash palindrome checker for constant-time queries after linear preprocessing.

Java Solution

```
var checker = new PalindromeSolutions.HashingQuerySolutions
    .RollingHashPalindromeChecker("racecar");
boolean result = checker.isPalindrome(1, 5);
```

Complexity

Preprocessing O(n), Query O(1), Space O(n).

Optimized Approach

Preprocessing plus O(1) queries is the optimized approach over checking each query range from scratch.

Question 87

Preprocess a string so each query `[l, r]` can be checked for palindrome status quickly.

Answer

Build prefix hashes for the original string and its reverse, together with the powers of the hash base. This preprocessing is what enables constant-time queries later.

Java Solution

```
var checker = new PalindromeSolutions.HashingQuerySolutions
    .RollingHashPalindromeChecker("abacaba");
```

Complexity

Preprocessing O(n), Space O(n).

Optimized Approach

This is itself the optimized approach for repeated queries.

Question 88

Use rolling hash to check whether substrings are palindromes.

Answer

Compute the forward hash of the substring and compare it with the corresponding backward hash from the reversed string.

Java Solution

```
boolean result = new PalindromeSolutions.HashingQuerySolutions
    .RollingHashPalindromeChecker("abacaba")
    .isPalindrome(2, 4);
```

Complexity

Preprocessing O(n), Query O(1), Space O(n).

Optimized Approach

In production, double hashing is often the optimized safer variant because it reduces collision risk.

Question 89

Support updates to characters and answer palindrome range queries.

Answer

The current mutable engine rebuilds the rolling-hash helper after each update, then answers queries in O(1).

Java Solution

```
var engine = new PalindromeSolutions.HashingQuerySolutions
    .MutablePalindromeQueryEngine("racecar");
```

```
engine.update(3, 'e');
boolean result = engine.isPalindrome(0, 6);
```

Complexity

Current implementation: Update O(n), Query O(1), Space O(n).

Optimized Approach

A more optimized approach uses Fenwick trees or segment trees with forward and reverse hashes so updates can be reduced to O(log n).

Question 90

Check whether a very large string is a palindrome when it cannot fit fully in memory.

Answer

Open the file with random access and compare one byte from the beginning with one byte from the end, moving inward without loading the full contents.

Java Solution

```
boolean result = PalindromeSolutions.HashingQuerySolutions
    .isLargeAsciiFilePalindrome(Path.of("huge.txt"));
```

Complexity

Time O(n), Space O(1).

Optimized Approach

Two-ended random access is the optimized strategy for large external data.

Question 91

Check whether a stream of incoming characters currently forms a palindrome.

Answer

Append the incoming characters to a growing buffer and test the current content whenever needed.

Java Solution

```
var stream = new PalindromeSolutions.HashingQuerySolutions.StreamPalindromeChecker();
stream.append('r');
stream.append('a');
stream.append('d');
stream.append('a');
stream.append('r');
boolean result = stream.isCurrentPalindrome();
```

Complexity

Append O(1) amortized, current palindrome check O(n), Space O(n).

Optimized Approach

An optimized approach for many queries would maintain rolling hashes incrementally rather than rescanning the whole current string each time.

Question 92

Design a data structure that supports append and palindrome-check operations.

Answer

Maintain the evolving string and a helper structure for palindrome queries. The current implementation stores the current characters and rebuilds the rolling hash helper on updates or appends.

Java Solution

```
var engine = new PalindromeSolutions.HashingQuerySolutions
    .MutablePalindromeQueryEngine("ab");
engine.append('a');
boolean result = engine.isPalindrome(0, engine.currentValue().length() - 1);
```

Complexity

Current implementation: Append O(n), Query O(1), Space O(n).

Optimized Approach

The optimized version is an online hash structure where append updates hashes incrementally in O(1) or O(log n).

13) Advanced Interview and Competitive Coding Questions

Question 93

Find the shortest palindrome by adding characters only at the front.

Answer

Find the longest palindromic prefix, then prepend the reverse of the leftover suffix.

Java Solution

```
String value = PalindromeSolutions.RearrangementConstructionSolutions
    .shortestPalindromeByAddingFront("abcd");
```

Complexity

Time O(n), Space O(n).

Optimized Approach

Using prefix-function/KMP logic to find the longest palindromic prefix is the optimized approach.

Question 94

Find the longest palindrome that can be formed from a multiset of characters.

Answer

A multiset view means only character counts matter. Use all even counts and place at most one odd-count character in the center.

Java Solution

```
int length = PalindromeSolutions.RearrangementConstructionSolutions
    .longestPossiblePalindromeLength("abccccdd");
```

Complexity

Time O(n log sigma), Space O(sigma).

Optimized Approach

Frequency counting is the optimized approach; there is no need to generate permutations.

Question 95

Find the maximum product of lengths of two disjoint palindromic subsequences.

Answer

Enumerate subsequences with bitmasks, keep the length of the masks that form palindromes, and combine disjoint masks for the best product.

Java Solution

```
int answer = PalindromeSolutions.AdvancedSolutions
    .maxProductOfTwoDisjointPalindromicSubsequences("leetcodecom");
```

Complexity

Time O(n * 2^n), Space O(2^n).

Optimized Approach

The implemented bitmask DP is already the optimized competitive-programming style solution for small n. The key optimization is pruning through subset DP instead of comparing every pair of subsequences naively.

Question 96

Count "super palindromes" where both the number and its square are palindromes.

Answer

Instead of testing every number, generate palindromic roots directly, square them, and check whether the square is also palindromic and falls in the requested range.

Java Solution

```
int count = PalindromeSolutions.AdvancedSolutions
    .countSuperPalindromes(1L, 100000L);
```

Complexity

Time depends on the number of palindromic roots up to sqrt(R); extra space O(1).

Optimized Approach

Generating palindromic roots directly is the optimized approach compared with scanning every integer in the interval.

Question 97

Find the largest palindromic number that can be formed from the digits of a string.

Answer

Count digit frequencies, build the left half from the largest digits first, reserve the largest leftover digit for the center, and mirror the left half.

Java Solution

```
String value = PalindromeSolutions.AdvancedSolutions
    .largestPalindromicNumber("444947137");
```

Complexity

Time O(n), Space O(1) extra beyond output.

Optimized Approach

This greedy counting approach is the optimized solution.

Question 98

Find the smallest palindromic number larger than a given numeric string.

Answer

Mirror the left half onto the right. If the result is not strictly larger than the input, increment the middle and propagate carry, then mirror again.

Java Solution

```
String value = PalindromeSolutions.AdvancedSolutions
    .smallestPalindromicNumberLargerThan("23545");
```

Complexity

Time O(n), Space O(n).

Optimized Approach

Mirroring plus middle carry propagation is the optimized classic approach.

Question 99

Determine whether a string can be split into exactly three palindromic substrings.

Answer

Precompute which substrings are palindromes, then try every possible first cut and second cut. If all three resulting pieces are palindromes, return true.

Java Solution

```
boolean result = PalindromeSolutions.AdvancedSolutions
    .canSplitIntoThreePalindromes("abcbdd");
```

Complexity

Time O(n^2), Space O(n^2).

Optimized Approach

The optimized approach is DP-based preprocessing rather than testing every partition with fresh palindrome scans.

Question 100

Find all ways to partition a string into exactly `k` palindromic parts.

Answer

Use the palindrome table for valid ranges, then backtrack while also tracking how many parts remain to be chosen.

Java Solution

```
List<List<String>> parts = PalindromeSolutions.AdvancedSolutions
    .partitionsIntoKPalindromes("aab", 2);
```

Complexity

Time O(n^2 + output_size), Space O(n^2 + output_size).

Optimized Approach

Precomputing palindrome ranges is the key optimization.

Question 101

Given a string, maximize palindrome length after at most `k` character changes.

Answer

Count how many mirrored pairs mismatch. Each allowed change can fix one mismatched pair, so the current implementation estimates how much of the string can be made palindromic.

Java Solution

```
int length = PalindromeSolutions.AdvancedSolutions
    .maxPalindromeLengthAfterAtMostKChanges("abcdef", 2);
```

Complexity

Time O(n), Space O(1).

Optimized Approach

For the specific goal of maximizing achievable palindrome length under this model, counting mismatches is the optimized direct approach.

Question 102

Given two strings, find the longest palindromic subsequence using characters from both.

Answer

Concatenate the strings and compute the LPS DP table, but only accept solutions whose left boundary comes from the first string and whose right boundary comes from the second.

Java Solution

```
int length = PalindromeSolutions.AdvancedSolutions
    .longestPalindromicSubsequenceUsingBothStrings("cacb", "cbba");
```

Complexity

Time O((n + m)^2), Space O((n + m)^2).

Optimized Approach

Using a combined DP table with cross-boundary validation is the optimized dynamic-programming strategy.

Question 103

Count palindromic paths in a tree or graph where labels are characters.

Answer

A common tree solution tracks a parity bitmask of character counts along the current path. A path can form a palindrome if at most one character has odd frequency, which means the final mask has at most one set bit.

Java Solution

```
int count = countPseudoPalindromicPaths(root, 0);
```

Complexity

Tree version typically runs in O(V + E) time with O(H) recursion stack. General graphs depend on traversal constraints and visited-state design.

Optimized Approach

The optimized approach is parity-bitmask tracking rather than storing the full multiset of path characters.

Question 104

Check whether a sentence remains a palindrome after applying a series of character updates.

Answer

Maintain the mutable sentence and answer full-range palindrome checks after each update. If the problem ignores punctuation or case, normalize the text before storing it.

Java Solution

```
var engine = new PalindromeSolutions.HashingQuerySolutions
    .MutablePalindromeQueryEngine("neveroddoreven");
engine.update(5, 'x');
boolean result = engine.isPalindrome(0, engine.currentValue().length() - 1);
```

Complexity

Current implementation: Update O(n), Query O(1), Space O(n).

Optimized Approach

An optimized version would use dynamic rolling hashes or segment trees to reduce update cost.

Question 105

Find the minimum operations needed to transform one string into a palindrome.

Answer

In this repository, operations are modeled as insertions, so the answer is computed through the minimum-insertions DP relation based on longest palindromic subsequence.

Java Solution

```
int ops = PalindromeSolutions.AdvancedSolutions
    .minOperationsToTransformToPalindrome("abcda");
```

Complexity

Time O(n^2), Space O(n^2).

Optimized Approach

The optimized answer depends on the exact operation set allowed. For insertions/deletions, the LPS-based DP is the standard optimization.

14) Popular Follow-Up Variations for Interviews

Question 106

Solve the basic palindrome problem first, then optimize for `O(1)` extra space.

Answer

Start from a straightforward solution, then refine it into the two-pointer method that compares the original sequence directly without auxiliary storage.

Java Solution

```
boolean result = PalindromeSolutions.InterviewFollowUps.isPalindromeO1Space("level");
```

Complexity

Time O(n), Space O(1).

Optimized Approach

The O(1)-space two-pointer approach is the optimized answer.

Question 107

Solve palindrome checking without using library helpers such as `reverse()`.

Answer

Do not depend on `reverse()` or `StringBuilder`; compare the input directly from both ends with indexes.

Java Solution

```
boolean result = PalindromeSolutions.InterviewFollowUps
    .isPalindromeNoLibraryHelpers("radar");
```

Complexity

Time O(n), Space O(1).

Optimized Approach

Direct index comparison is the optimized and most interview-friendly approach.

Question 108

Solve the same problem for mutable input such as `char[]`.

Answer

The logic is unchanged: compare the left and right ends and move inward until the center is reached.

Java Solution

```
boolean result = PalindromeSolutions.InterviewFollowUps
    .isPalindromeMutable(new char[]{'n', 'o', 'o', 'n'});
```

Complexity

Time O(n), Space O(1).

Optimized Approach

The optimized answer is still the in-place two-pointer scan.

Question 109

Explain the trade-offs between iterative, recursive, stack-based, and stream-based solutions.

Answer

Iterative two pointers are usually best because they are simple and O(1) space. Recursive solutions are elegant but consume call-stack memory. Stack-based solutions are intuitive but use extra space. Stream-based versions are expressive but often less direct and not usually the fastest.

Java Solution

```
boolean iterative = PalindromeSolutions.BasicStringSolutions.isPalindrome("level");
boolean recursive = PalindromeSolutions.BasicStringSolutions.isPalindromeRecursive("level");
boolean stack = PalindromeSolutions.BasicStringSolutions.isPalindromeWithStack("level");
boolean stream = PalindromeSolutions.ConstraintStringSolutions.isPalindromeStream("level");
```

Complexity

Iterative O(n)/O(1), Recursive O(n)/O(n), Stack O(n)/O(n), Stream O(n)/O(1) extra.

Optimized Approach

The optimized practical answer for most interviews is the iterative two-pointer solution.

Question 110

Modify the solution to ignore case, spaces, punctuation, or all non-alphanumeric characters.

Answer

Normalize the comparison rules while scanning instead of changing the core palindrome idea. Skip irrelevant characters and lowercase letters before comparing them.

Java Solution

```
boolean result = PalindromeSolutions.InterviewFollowUps
    .isPalindromeIgnoringNoise("A man, a plan, a canal: Panama");
```

Complexity

Time O(n), Space O(1).

Optimized Approach

The optimized version ignores noise during the scan instead of allocating a cleaned copy first.

Question 111

Extend the solution from strings to arrays, lists, and linked lists.

Answer

The palindrome concept never changes: compare mirrored positions. What changes is how those positions are accessed in each data structure.

Java Solution

```
boolean arrayOk = PalindromeSolutions.ArrayCollectionSolutions
    .isArrayPalindrome(new int[]{1, 2, 1});
boolean listOk = PalindromeSolutions.ArrayCollectionSolutions
    .isListPalindrome(List.of("x", "y", "x"));
boolean linkedOk = PalindromeSolutions.LinkedListSolutions.isPalindromeAndRestore(
    PalindromeSolutions.LinkedListSolutions.SinglyNode.of(1, 2, 1)
);
```

Complexity

Arrays/lists O(n), linked lists O(n); extra space depends on the linked-list technique used.

Optimized Approach

Optimized array/list solutions use two pointers, while optimized linked-list solutions reverse the second half instead of using a stack.

Question 112

Optimize from brute force to dynamic programming for substring and subsequence problems.

Answer

Brute force often checks every possible substring, partition, or subsequence and becomes too slow. For substrings, center expansion avoids redundant work. For subsequences and partitioning, dynamic programming stores results for overlapping subproblems and reuses them.

Java Solution

```
String bestSubstring = PalindromeSolutions.SubstringSolutions
    .longestPalindromicSubstring("babad");
int bestSubsequence = PalindromeSolutions.DynamicProgrammingSolutions
    .longestPalindromicSubsequenceLength("bbbab");
```

Complexity

Brute-force substring methods are often O(n^3) or worse; optimized substring and DP subsequence methods are typically O(n^2).

Optimized Approach

The key optimized idea is reuse: center expansion for substrings, DP tables for subsequences and partitioning.

Question 113

Discuss edge cases such as empty strings, single characters, null input, negative numbers, and overflow.

Answer

Good palindrome solutions define what happens for empty input, one-character input, and nulls. Numeric variants should reject negative numbers under the usual signed-number rule and protect reversal logic against overflow.

Java Solution

```
boolean empty = PalindromeSolutions.BasicStringSolutions.isPalindrome("");
boolean single = PalindromeSolutions.BasicStringSolutions.isPalindrome("a");
boolean negative = PalindromeSolutions.NumberPalindromeSolutions.isPalindromeWithNegativeRule(-121);
OptionalInt reversed = PalindromeSolutions.NumberPalindromeSolutions.reverseIntSafely(Integer.MAX_VALUE);
```

Complexity

Edge handling adds only O(1) overhead beyond the main algorithm.

Optimized Approach

The optimized mindset is to reject impossible cases early and guard risky operations such as numeric reversal.